

プログラミング演習スライド

金子知適

東京大学総合文化研究科 (kaneko@acm.org)

week5

再帰 recursion

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

再帰の位置づけ

- 複雑さを管理下に置く道具の1つ
e.g., フラクタル, グラフの探索
- 反復 (for, while) と補完し合う

再帰と反復との関係

	反復得意	反復苦手
再帰得意	(1)	(2)
再帰苦手	(3)	-

再帰でも反復でもかける内容 (1) から始めて, 再帰が得意とする内容 (2) に進む

再帰関数の基本パターン — 漸化式と対応させる —

例: $n \geq 0$ までの和
漸化式

$$\text{sum}(n) = \begin{cases} 0 & n = 0 \text{ のとき} \\ n + \underbrace{\text{sum}(n-1)}_{n-1 \text{ までの和}} & \text{それ以外} \end{cases}$$

```
def sumr(n):  
    if n == 0: # basecase  
        return 0  
    else: # recursion step  
        return n + sumr(n-1)  
                                n-1までの和
```

(sumr の r は recursion から命名)

再帰関数の計算の実際

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

再帰関数

```
def sumr(n):  
    if n == 0: # basecase  
        return 0  
    else: # recursion step  
        return n + sumr(n-1)  
                    n-1までの和
```

```
1 def sumr(n=3):  
2     if 3 == 0: # False!  
3         return 0  
4     else:  
5         return 3 + sumr(3-1)
```

```
>>> sumr(3)
```

1

再帰関数の計算の実際

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

再帰関数

```
def sumr(n):  
    if n == 0: # basecase  
        return 0  
    else: # recursion step  
        return n + sumr(n-1)  
                    n-1までの和
```

```
>>> sumr(3)
```

1

```
1 def sumr(n=3):  
2     if 3 == 0: # False!
```

```
3  
4  
5  
1 def sumr(n=2):  
2     if 2 == 0: # False!  
3         return 0  
4     else:  
5         return 2 + sumr(2-1)
```

再帰関数の計算の実際

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

再帰関数

```
def sumr(n):  
    if n == 0: # basecase  
        return 0  
    else: # recursion step  
        return n + sumr(n-1)  
                    n-1までの和
```

```
>>> sumr(3)
```

1

```
1 def sumr(n=3):  
2     if 3 == 0: # False!
```

```
3  
4 1 def sumr(n=2):  
2     if 2 == 0: # False!
```

```
3  
4 1 def sumr(n=1):  
2     if 1 == 0: # False!  
3         return 0  
4     else:  
5         return 1 + sumr(1-1)
```

再帰関数の計算の実際

プログラミング演習
スライド

金子

再帰
線形再帰
枝分かれを伴う再帰

再帰関数

```
def sumr(n):  
    if n == 0: # basecase  
        return 0  
    else: # recursion step  
        return n + sumr(n-1)  
                    n-1までの和
```

```
>>> sumr(3)
```

1

```
1 def sumr(n=3):  
2     if 3 == 0: # False!
```

```
3  
4 1 def sumr(n=2):  
5     if 2 == 0: # False!
```

```
3  
4 1 def sumr(n=1):  
5     if 1 == 0: # False!
```

```
3  
4 1 def sumr(n=0):  
5     if 0 == 0: # True!  
3         return 0  
4     else:  
5         return 1 + sumr(0-1)
```

再帰関数の計算の実際

プログラミング演習
スライド

金子

再帰
線形再帰
枝分かれを伴う再帰

再帰関数

```
def sumr(n):  
    if n == 0: # basecase  
        return 0  
    else: # recursion step  
        return n + sumr(n-1)  
                    n-1までの和
```

```
>>> sumr(3)
```

1

```
1 def sumr(n=3):  
2     if 3 == 0: # False!
```

```
3  
4  
5  
1 def sumr(n=2):
```

```
2  
3  
4  
5  
1 def sumr(n=1):  
2     if 1 == 0: # False!  
3         return 0  
4     else:  
5         return 1 + sumr(1-1)  
                    0  
                    1
```

再帰関数の計算の実際

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

再帰関数

```
def sumr(n):  
    if n == 0: # basecase  
        return 0  
    else: # recursion step  
        return n + sumr(n-1)  
                    n-1までの和
```

```
>>> sumr(3)
```

1

```
1 def sumr(n=3):
```

```
2     1 def sumr(n=2):
```

```
3         2 if 2 == 0: # False!
```

```
4             3 return 0
```

```
5         4 else:
```

```
5             return 2 + sumr(2-1)  
                        1  
                        3
```


再帰関数の計算の実際

プログラミング演習
スライド

金子

再帰
線形再帰
枝分かれを伴う再帰

再帰関数

```
def sumr(n):  
    if n == 0: # basecase  
        return 0  
    else: # recursion step  
        return n + sumr(n-1)  
                    n-1までの和
```

```
>>> sumr(3)
```

1

```
1 def sumr(n=3):  
2     if 3 == 0: # False!  
3         return 0  
4     else:  
5         return 3 + sumr(3-1)  
                    3  
                    6
```

再帰関数の計算の実際

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

再帰関数

```
def sumr(n):  
    if n == 0: # basecase  
        return 0  
    else: # recursion step  
        return n + sumr(n-1)  
                    n-1までの和
```

```
>>> sumr(3)      1  
6                2
```

評価 (実行) 完了

再帰の注意

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

Maximum recursion depth exceeded

再帰関数は、basecase で止まるように実装する必要がある。
以下はダメな例で、無限再帰する

```
def inf():  
    # basecase がない!  
    return 1 + inf() # step
```

この関数をinf()と呼び出すと、

```
>>> inf()  
Traceback (most recent call last):  
...  
[Previous line repeated 995 more times]  
RecursionError: maximum recursion depth exceeded
```

というように、RecursionError: maximum recursion depth exceeded というエラーで処理を続行できなくなった旨表示される。

複数の表現例

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

再帰関数の基本パターン — 漸化式と対応させる —

例: $n \geq 0$ までの和 (再掲) —
漸化式

$$\text{sum}(n) = \begin{cases} 0 & n = 0 \text{ のとき} \\ n + \underbrace{\text{sum}(n-1)}_{n-1 \text{ までの和}} & \text{それ以外} \end{cases}$$

```
def sumr(n):  
    if n == 0: # basecase  
        return 0  
    else: # recursion step  
        return n + sumr(n-1)  
                        n-1までの和
```

例: $n \geq 0$ までの和 別表現 (発展 — 末尾再帰) —
漸化式

$\text{sum}(n) = \text{sum}'(0, n)$, where

$$\text{sum}'(s, n) = \begin{cases} s & n = 0 \text{ のとき} \\ \text{sum}'(s+n, n-1) & \text{それ以外} \end{cases}$$

変数 s は, 「大きい方からの和」

```
def sum2(s, n):  
    if n == 0: # basecase  
        return s  
    else: # recursion step  
        return sum2(s+n, n-1)  
                        (s+n)+(n-1までの和)
```

例題

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

例: 最大公約数
漸化式

$$\text{gcd}(a, b) = \begin{cases} b & \text{if } a \text{ is a multiple of } b \\ \text{gcd}(b, a \% b) & \text{otherwise} \end{cases}$$

実用 `math.gcd(a, b)`

```
def gcd(a, b):  
    if a % b == 0: # basecase  
        return b  
    else:         # step  
        return gcd(b, a % b)
```

例: 平方根の二分法 (「情報」)
漸化式

$$\text{sqrt}(x, a, b) = \begin{cases} m & \text{if } |m^2 - x| < \epsilon \\ \text{sqrt}(x, \underbrace{a, m}_{\text{左半分}}) & \text{if } m^2 > x \\ \text{sqrt}(x, \underbrace{m, b}_{\text{右半分}}) & \text{otherwise} \end{cases}$$

実用 `math.sqrt(x)`

```
Epsilon = 0.0001 # 許容誤差  
def sqrt(x, a, b):  
    m = (a + b) / 2.0  
    if abs(x - m * m) <  
        ↪ Epsilon:  
        return m # basecase  
    elif m * m > x: # step  
        return sqrt(x, a, m)  
    else:          # step  
        return sqrt(x, m, b)
```

再帰的手続き

手続き (値を返さない関数, 副作用で仕事) を再帰的に書くこともできる. 漸化式とはもはや対応しないが, basecase と step という考え方は有用である.

— nから1まで表示 —

```
def printnum(n):  
    if n == 0: # ~ basecase  
        return  
    else: # ~ step  
        # ここまでに n < x の x は表示済み  
        print(n)  
        printnum(n-1)
```

```
>>> printnum(3)          1  
3                          2  
2                          3  
1                          4
```

— 1からnまで表示 —

```
def printnum_inc(cur, n):  
    if cur > n: # ~ basecase  
        return  
    else: # ~ step  
        # ここまでに x < cur の x は表示済み  
        print(cur)  
        printnum_inc(cur+1, n)
```

```
>>> printnum_inc(1, 3)   1  
1                          2  
2                          3  
3                          4
```

反復と再帰 — 直線的な再帰は反復と対応

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

例: $n \geq 0$ までの和

漸化式

$$\text{sum}(n) = \begin{cases} 0 & n = 0 \text{ のとき} \\ n + \underbrace{\text{sum}(n-1)}_{\text{\textcolor{red}{n-1までの和}}} & \text{それ以外} \end{cases}$$

再帰と反復の関係 (再掲)

	反復得意	反復苦手
再帰得意	(1) 線形再帰	(2)
再帰苦手	(3)	-

再帰版 (再掲)

```
def sumr(n):  
    if n == 0:  
        return 0  
    else:  
        return n + sumr(n-1)  
                        \text{\textcolor{red}{n-1までの和}}
```

等価な for の実装

```
def sumf(n):  
    total = 0  
    for i in range(n+1):  
        # (a1) この時点で total は 0 (i == 0)  
        # (a2) この時点で total は i-1 までの和  
        total += i  
        # (b) この時点で total は i までの和  
    return total
```

反復と再帰 — 直線的な再帰は反復と対応

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

例: 平方根の二分法 (「情報」)

漸化式

$$\text{sqrt}(x, a, b) = \begin{cases} m & \text{if } |m^2 - x| < \epsilon \\ \text{sqrt}(x, a, m) & \text{if } m^2 > x \\ \text{sqrt}(x, m, b) & \text{otherwise} \end{cases}$$

再帰と反復の関係 (再掲)

	反復得意	反復苦手
再帰得意	(1) 線形再帰	(2)
再帰苦手	(3)	-

再帰版 (再掲)

```
Epsilon = 0.0001 # 許容誤差
def sqrt(x, a, b):
    m = (a + b) / 2.0
    if abs(x - m * m) <
        ↪ Epsilon:
        return m
    elif m * m > x:
        return sqrt(x, a, m)
    else:
        return sqrt(x, m, b)
```

左半分

右半分

反復版

```
def sqrt_iter(x, a, b):
    m = (a + b) / 2.0
    while abs(x - m * m) >= Epsilon:
        if m * m > x:
            b = m # 次の区間[a,b] ← 今の左半分[a,m]
        else:
            a = m # 次の区間[a,b] ← 今の右半分[m,b]
        m = (a + b) / 2.0
    return m
```


枝分かれを伴う再帰

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

いろいろな再帰

	関数	手続き
易 線形	sum(n) gcd(a,b), sqrt(x,a,b)	printnum(n)
難 枝分かれ 合流無し	木, fractal	カメ fractal, merge sort
難 枝分かれ 合流あり	fibonacci , nCm	一般迷路

枝分かれの例 — 木[‡]の探索 —

```
0 #####
1 #.##....##
2 #.##.#####
3 #@.....####
4 #####
0123456789
```

Ascii-Maze.ipynb

右と上方向だけ進める迷路。 $(r, c) = (3, 1)$ から始める
到達可能なマス $C(3, 1)$ は何マス?

$$C(r, c) = \begin{cases} 0 & (r, c) \text{ が壁のとき} \\ \underbrace{1}_{\text{現在地}} + \underbrace{C(r, c+1)}_{\text{右}} + \underbrace{C(r-1, c)}_{\text{上}} & \text{それ以外} \end{cases}$$

[‡] 位置をノード，接続関係を辺とするグラフで表現すると，木構造に対応

枝分かれを伴う再帰

プログラミング演習
スライド

金子

再帰

線形再帰

枝分かれを伴う再帰

いろいろな再帰

	関数	手続き
易 線形	sum(n) gcd(a,b), sqrt(x,a,b)	printnum(n)
難 枝分かれ 合流無し	木, fractal	カメ fractal, merge sort
難 枝分かれ 合流あり	fibonacci , nCm	一般迷路

枝分かれの例 — 木[‡]の探索 —

```
0 #####
1 #3##..7.##
2 #.##.#####
3 #@..1.####
4 #####
  0123456789
```

Ascii-Maze.ipynb

右と上方向だけ進める迷路. $(r, c) = (3, 1)$ から始める
拾える**最高の宝** $T(3, 1)$ は?

$$T(r, c) = \begin{cases} 0 & (r, c) \text{ が壁のとき} \\ \max \left(\underbrace{n}_{\text{現在地の宝}}, \underbrace{T(r, c+1)}_{\text{右}}, \underbrace{T(r-1, c)}_{\text{上}} \right) & \text{それ以外} \end{cases}$$

[‡] 位置をノード, 接続関係を辺とするグラフで表現すると, 木構造に対応

Fractal — Koch curve

(鑑賞だけで良い)

プログラミング演習
スライド

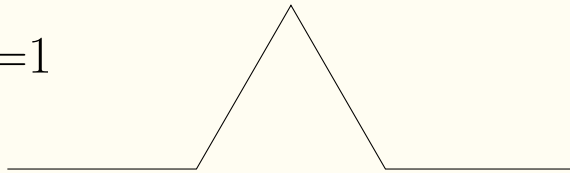
金子

再帰
線形再帰
枝分かれを伴う再帰

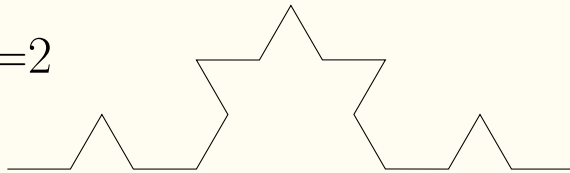
n=0



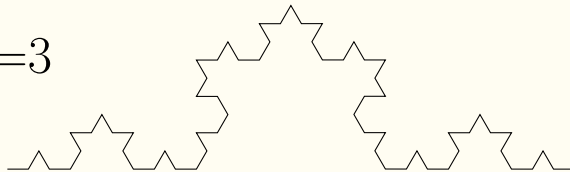
n=1



n=2



n=3



再帰構造の構成

$n \rightarrow n + 1$ で各辺を 4 分割. 各辺は相似.

作成方針

$K_n(s, t)$ $\doteq$ 点 s から点 t に n 段の Koch 曲線を描く各辺の始点の列[†]

$m_1, m_2, m_3 \doteq$ 辺を 4 分割する頂点 a

$$K_n(s, t) = \begin{cases} [s] & n = 0 \\ K_{n-1}(s, m_1) + K_{n-1}(m_1, m_2) \\ \quad + K_{n-1}(m_2, m_3) + K_{n-1}(m_3, t)^\ddagger & n > 0 \end{cases}$$

[†] 重複防止: 終点=次の始点なので, s を含み, t を含まない

[‡] この + 演算はリストの連結

中点

$cx, cy = (sx+tx)/2, (sy+ty)/2$

進行方向

$vx, vy = tx-sx, ty-sy$

$vx2, vy2 = -vy, vx$ # 90度左回転

$c = \text{math.sqrt}(3)/6$

中点から左手に c 倍 伸ばす

$m2x, m2y = cx+vx2*c, cy+vy2*c$

1/3, 2/3 の内分点

$m1x, m1y = (2*sx+tx)/3, (2*sy+ty)/3$

2/3, 1/3 の内分点

$m3x, m3y = (sx+2*tx)/3, (sy+2*ty)/3$

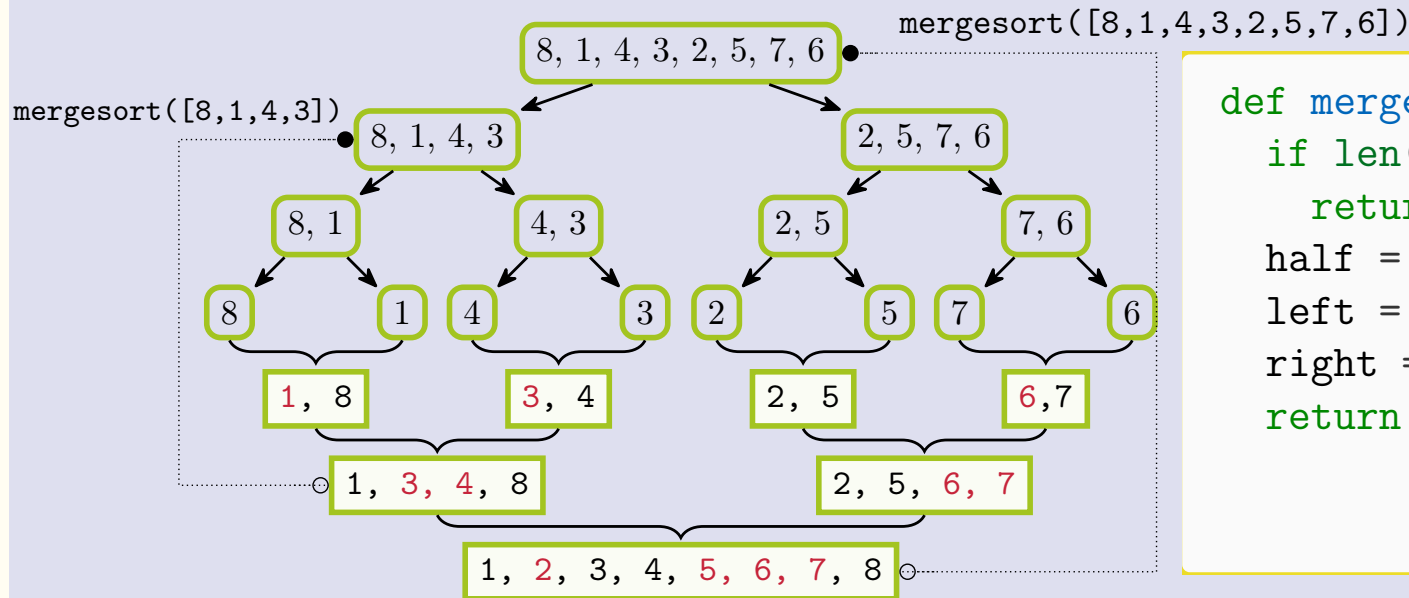
Merge sort

プログラミング演習
スライド

金子

再帰
線形再帰
枝分かれを伴う再帰

再帰構造 — 左半分の整列と右半分の整列の合併



```
def mergesort(seq):  
    if len(seq) <= 1: # basecase  
        return seq  
    half = len(seq)//2  
    left = seq[0:half]  
    right = seq[half:len(seq)]  
    return merge(  
        mergesort(left),  
        mergesort(right) )
```

新文法 **スライス** 配列[式左:式右]

`seq=[8,1,4,3,2,5,7,6]`

```
def mergesort(seq):  
    ...  
    half = len(seq)//2  
    left = seq[0:half]  
    right = seq[half:len(seq)]  
    return merge(mergesort(left), mergesort(right))
```

Annotations for the left half (`seq=[8,1,4,3]`):

- `half = len(seq)//2`: `4`
- `left = seq[0:half]`: `[8,1,4,3]`
- `right = seq[half:len(seq)]`: `[2,5,7,6]`
- `mergesort(left)`: `[1,3,4,8]`
- `mergesort(right)`: `[2,5,6,7]`

`seq=[8,1,4,3]`

```
def mergesort(seq):  
    ...  
    half = len(seq)//2  
    left = seq[0:half]  
    right = seq[half:len(seq)]  
    return merge(mergesort(left), mergesort(right))
```

Annotations for the left half (`seq=[8,1]`):

- `half = len(seq)//2`: `2`
- `left = seq[0:half]`: `[8,1]`
- `right = seq[half:len(seq)]`: `[4,3]`
- `mergesort(left)`: `[1,8]`
- `mergesort(right)`: `[3,4]`

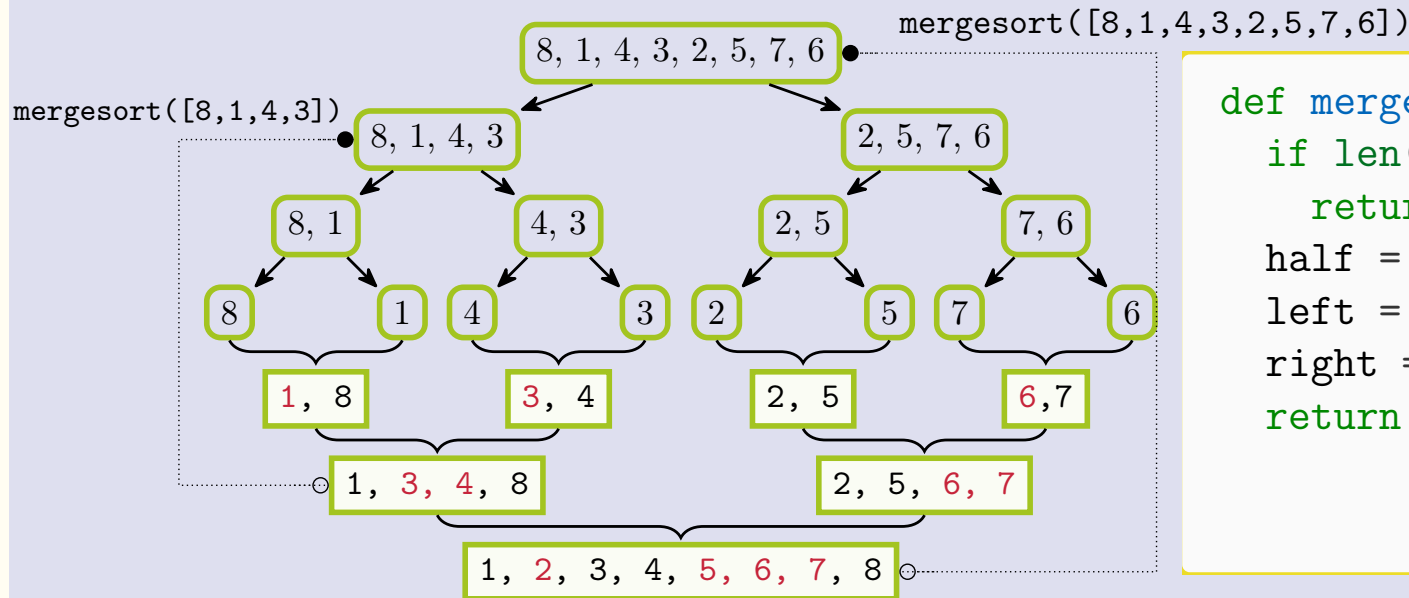
Merge sort

プログラミング演習
スライド

金子

再帰
線形再帰
枝分かれを伴う再帰

再帰構造 — 左半分の整列と右半分の整列の合併

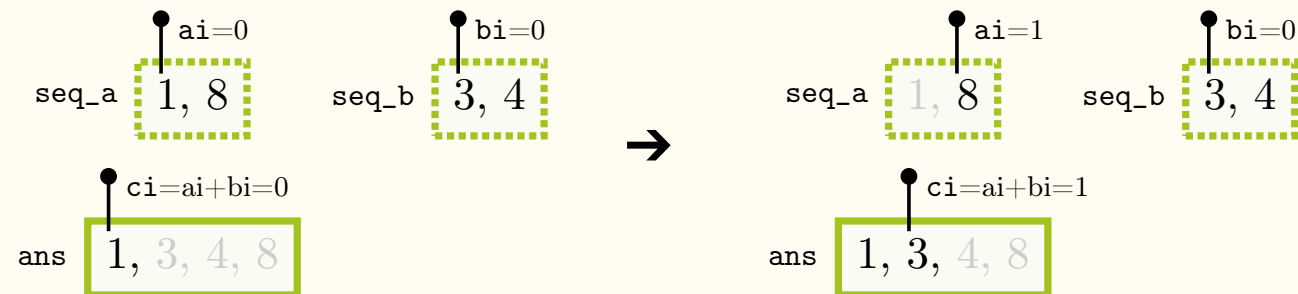


```
def mergesort(seq):  
    if len(seq) <= 1: # basecase  
        return seq  
    half = len(seq)//2  
    left = seq[0:half]  
    right = seq[half:len(seq)]  
    return merge(  
        mergesort(left),  
        mergesort(right) )
```

新文法 **スライス** 配列[式左:式右]

■ 実装 merge(left, right) があれば簡単

最初のステップの例



- 漸近計算量 $O(N \log N)$ (配列長 N)
水平に見ると各行で $O(N)$ の処理, 全体で $O(\log N)$ 行